CSCI 232 Assignment 05 – Linked List in C++ & C/C++ Integration

Points: 10

Instructions

1. **Fork** the repository on GitHub: `https://github.com/jpach-cs/232-ass05-template`
2. **Clone** your forked repository to your local machine.
3. Open `src/code.cpp` and `src/tests.cpp` and `include/code.hpp`.
4. Uncomment the author block at the top of `src/code.cpp` and fill in your name:

```
// std::string AUTHOR_NAME       = "Your Name";
// std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this
// assignment independently...";
```

5. Work through each function and class method in **two parts**:

Part 1 — Legacy Data & Node Integration (`src/code.cpp`)

- Implement `printLegacyData` to safely handle legacy C unions in C++.
- Implement node initialization and helper creation functions for `struct`, `classNodeVariant`, and template nodes.
- Do **not** modify function signatures in `include/code.hpp`.

Part 2 — `LinkedList` Class Implementation (`src/code.cpp`)

- Implement RAII memory management in the constructor and destructor.
- Implement member methods for insertion, deletion, searching, and traversal.
- Ensure all methods handle `nullptr` guards and empty list edge cases.

Part 3 — Unit Tests (`src/tests.cpp`)

- Implement unit test bodies using `TEST_ASSERT_`.
- Clean up any dynamically allocated memory in test cases to prevent leaks.

1. Build and run locally using C++20 (`g++`):

```
make clean
make
./run_tests
```

7. When all local tests pass — **commit** and **push** to your repository.
8. Go to the **Actions** tab on GitHub and verify the CI results.

Functions — `src/code.cpp` (Part 1)

Function	Description
`printLegacyData`	Format `LegacyData` union value (`int`, `double`, `char*`) into `std::string`
`initStructNode`	Initialize a raw `StructNode` with a value and clear `next`
`createTwoStructNodes`	Allocate and w-link two `StructNode` instances on the heap
`createTwoClassNodes`	Allocate and w-link two `classNodeVariant` objects on the heap
`createTwoTemplateNodes`	Instantiate and link two generic `TemplateNode<T>` objects

`LinkedList` Class Methods — `src/code.cpp` (Part 2)

Method	Description
`LinkedList()`	Constructor — initialize empty list (`head = nullptr`, `count = 0`)
`~LinkedList()`	Destructor — free all allocated nodes in the list (RAII)
`addFirst`	Insert `classNodeVariant*` at the front of the list (with `nullptr` guard)
`addLast`	Insert `classNodeVariant*` at the end of the list (with `nullptr` guard)
`deleteFirst`	Remove and free the first node; update `head` and `count`
`deleteLast`	Remove and free the last node; update pointers and `count`
`deleteValue`	Search, remove, and free the first node matching `NodeValue`
`printList`	Print all node values to `std::cout` using `std::visit`; return count
`listLength`	Return the current number of nodes in the list

Tests to implement — `src/tests0.cpp`

Test function	What to verify
`test_printLegacyData_int`	Formats integer field `'i'` into valid `std::string`
`test_printLegacyData_double`	Formats double field `'d'` rounded to 2 decimal places
`test_printLegacyData_string`	Formats `char*` string or returns `"nullptr"` if NULL
`test_printLegacyData_null_ptr`	Safely handles NULL string pointer without crashing
`test_printLegacyData_unknown_type`	Returns `"unknown"` for unsupported type char
`test_initStructNode_sets_fields`	Sets value correctly and sets `next` to `nullptr`
`test_linkedList_addFirst_updates_addFirst`	`addFirst` updates `head` and increments `count`
`test_linkedList_addFirst_null_guard`	`addFirst(nullptr)` does not increment counter or crash
`test_linkedList_addLast_null_guard`	`addLast(nullptr)` handles NULL input safely
`test_linkedList_deleteFirst_empty`	`deleteFirst()` on empty list does not crash
`test_linkedList_deleteLast_empty`	`deleteLast()` on empty list handles edge case safely
`test_linkedList_deleteValue_not_found`	`deleteValue()` with missing value leaves list unchanged
`test_linkedList_printList_empty`	`printList()` on empty list returns 0

Rules

- Do **not** modify headers or function **signatures** in `inc/code.hpp`.
- All code must compile under **C++20** (`-std=c++20`).
- Always guard against `nullptr` when dereferencing pointers.
- Always clean up dynamically allocated nodes (`delete`) in tests and destructor.
- CI runs your `tests05.cpp` together with instructor validation tests — both must pass.

Grading

Component	Points
Legacy integration & node creation functions	3
`LinkedList` class implementation (RAII & methods)	4
All test cases in `tests05.cpp` passing	3
Total	**10**

Author block — required

```
std::string AUTHOR_NAME       = "Your Name";
std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this "
"assignment independently, except where explicitly noted and referenced. "
"Any collaboration or use of external resources has been properly cited. "
"I am fully aware of the consequences of academic dishonesty and agree "
"to abide by the university's academic integrity policy.";
```